VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Mahima Koul (1BF24CS161)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Mahima Koul **(1BF24CS161)**, who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

LAB PROGRAM 1:

a) Write a program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>
#define MAX 20

int graph[MAX][MAX], indegree[MAX], queue[MAX];
int front = -1, rear = -1, n;

void enqueue(int item) {
   if (rear == MAX - 1)
      return;
   if (front == -1)
      front = 0;
   queue[++rear] = item;
}

int dequeue() {
   if (front == -1 || front > rear)
      return -1;
   return queue[front++];
}

void topologicalSort() {
   int i, j, count = 0, node;

   for (i = 0; i < n; i++) {
      indegree[i] = 0;
      for (j = 0; j < n; j++) {
         if (graph[j][i] == 1)
            indegree[i]++;
      }
   }
   for (i = 0; i < n; i++) {
      if (indegree[i] == 0)
         enqueue(i);
   }

   printf("Topological Ordering: ");

   while (front <= rear) {
      node = dequeue();
      printf("%d ", node);
      count++;

      for (i = 0; i < n; i++) {
         if (graph[node][i] == 1) {
            indegree[i]--;
            if (indegree[i] == 0)
               enqueue(i);
```

```
            }
        }
    }

    if (count != n)
        printf("\nGraph contains a cycle. Topological ordering not possible.");
}

int main() {
    int i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    topologicalSort();
    return 0;
}
```

Output:

```
Enter number of vertices: 6
Enter adjacency matrix:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological Ordering: 0 1 2 3 4 5
Process returned 0 (0x0)   execution time : 48.183 s
Press any key to continue.
```

b) Leetcode Program related to Topological Sorting

Leetcode Problem 207: Course Schedule

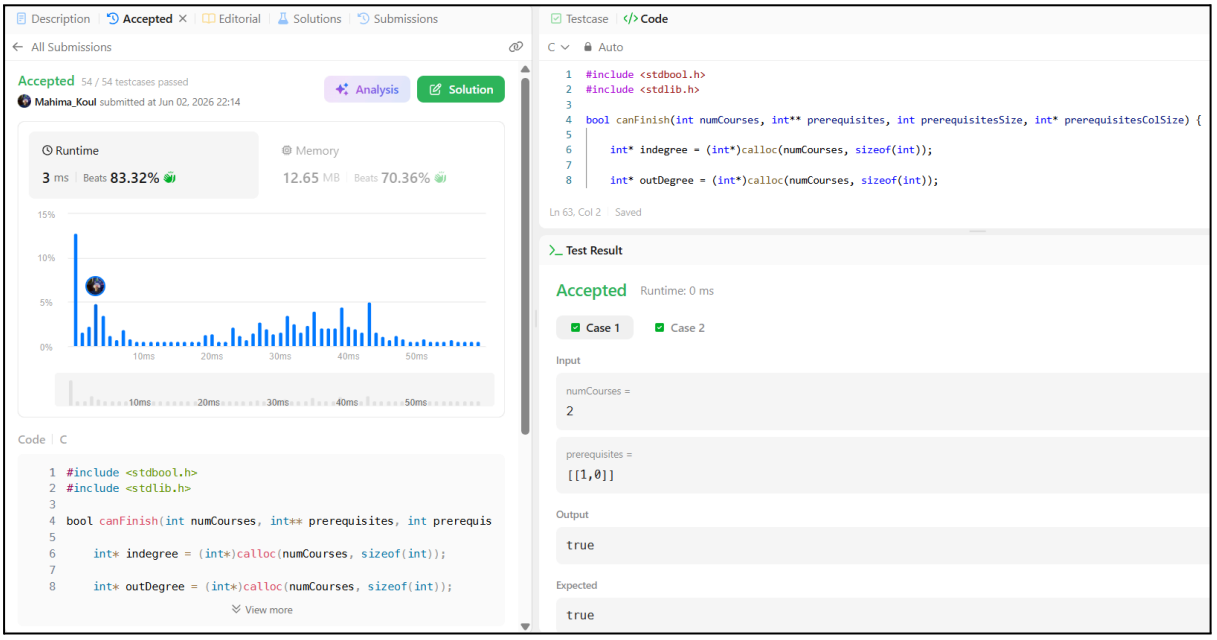
Q: There are a total of numCourses courses you have to take, labeled from 0 to numCourses − 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

Code:

```
#include <stdbool.h>
#include <stdlib.h>

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int* outDegree = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        outDegree[b]++;
        indegree[a]++;
    }

    int** graph = (int**)malloc(numCourses * sizeof(int*));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(outDegree[i] * sizeof(int));
        outDegree[i] = 0;
    }

    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][outDegree[b]++] = a;
    }

    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    int completed = 0;
    while (front < rear) {
        int course = queue[front++];
        completed++;

        for (int i = 0; i < outDegree[course]; i++) {
            int next = graph[course][i];
            indegree[next]--;

            if (indegree[next] == 0) {
                queue[rear++] = next;
            }
```

```
        }
    }

    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }
    free(graph);
    free(indegree);
    free(outDegree);
    free(queue);
    return completed == numCourses;
}
```

Output:



LAB PROGRAM 2:

Implement Johnson Trotter Algorithm to generate permutations

Code:

```
#include <stdio.h>
#define LEFT 0
#define RIGHT 1

int getMobile(int a[], int dir[], int n)
{
   int mobile = 0;
   int mobileIndex = -1;
   for (int i = 0; i < n; i++)
   {
      if (dir[i] == LEFT && i != 0 && a[i] > a[i - 1])
      {
         if (a[i] > mobile)
         {
            mobile = a[i];
            mobileIndex = i;
         }
      }
      if (dir[i] == RIGHT && i != n - 1 && a[i] > a[i + 1])
      {
         if (a[i] > mobile)
         {
            mobile = a[i];
            mobileIndex = i;
         }
      }
   }
   return mobileIndex;
}

void printPermutation(int a[], int n)
{
   for (int i = 0; i < n; i++)
      printf("%d ", a[i]);
   printf("\n");
}

void johnsonTrotter(int n)
{
   int a[n];
   int dir[n];
   for (int i = 0; i < n; i++)
   {
      a[i] = i + 1;
      dir[i] = LEFT;
   }
   printPermutation(a, n);
   while (1)
```

```
    {
        int mobileIndex = getMobile(a, dir, n);
        if (mobileIndex == -1)
            break; // No mobile element left
        int mobile = a[mobileIndex];
        if (dir[mobileIndex] == LEFT)
        {
            int temp = a[mobileIndex];
            a[mobileIndex] = a[mobileIndex - 1];
            a[mobileIndex - 1] = temp;
            int dtemp = dir[mobileIndex];
            dir[mobileIndex] = dir[mobileIndex - 1];
            dir[mobileIndex - 1] = dtemp;
            mobileIndex--;
        }
        else
        {
            int temp = a[mobileIndex];
            a[mobileIndex] = a[mobileIndex + 1];
            a[mobileIndex + 1] = temp;
            int dtemp = dir[mobileIndex];
            dir[mobileIndex] = dir[mobileIndex + 1];
            dir[mobileIndex + 1] = dtemp;
            mobileIndex++;
        }

        for (int i = 0; i < n; i++)
        {
            if (a[i] > mobile)
                dir[i] = !dir[i];
        }
        printPermutation(a, n);
    }
}

int main()
{
    int n;
    printf("Enter value of n: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}
```

Output:

```
Enter value of n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 1.318 s
Press any key to continue.
```

```
Enter value of n: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

Process returned 0 (0x0)   execution time : 1.501 s
Press any key to continue.
```

LAB PROGRAM 3:

a)Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

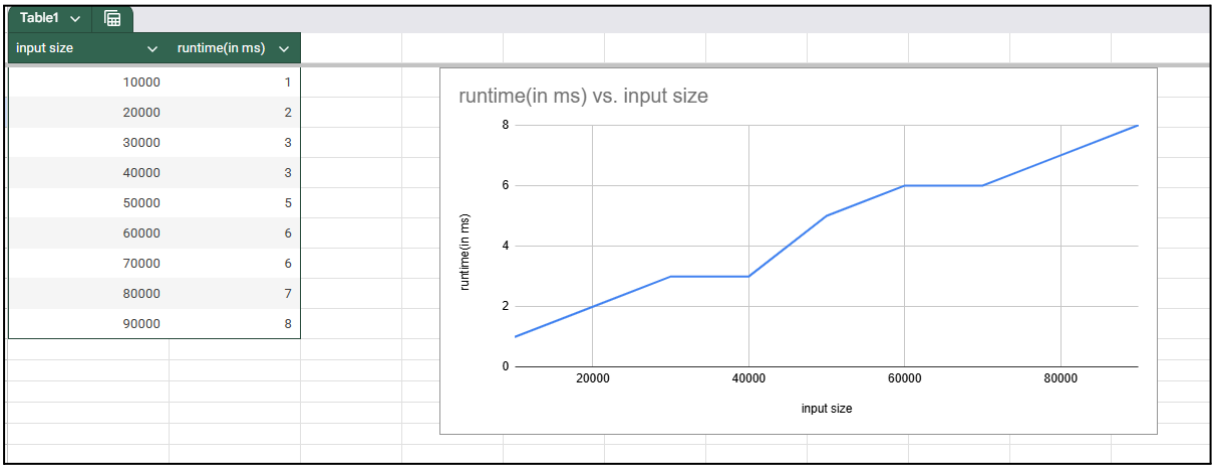
```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
int c[100000];

void merge(int low, int mid, int high, int a[])
{
   int i= low;
   int j= mid+1;
   int k= low;

   while(i<= mid && j<= high)
   {
      if(a[i]< a[j])
      {
         c[k]= a[i];
         k++;
         i++;
      }
      else
      {
         c[k]= a[j];
         k++;
         j++;
      }
   }
   while(i<= mid){
      c[k]=a[i];
      k++;
      i++;
   }
   while(j<= high){
      c[k]=a[j];
      k++;
      j++;
   }
   for(int m= low; m<= high; m++){
      a[m]=c[m];
   }
}

void merge_sort(int low, int high, int a[])
{
   if(low<high)
   {
      int mid=(low+high)/2;
```

```
        merge_sort(low, mid,a);
        merge_sort(mid+1, high, a);
        merge(low, mid, high, a);
    }
}

int main()
{
    int a[100000], n, i;
    clock_t start, end;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    for(int i=0; i<n; i++){
        printf("\nEnter element %d: ", i+1);
        scanf("%d", &a[i]);
    }
    start=clock();
    merge_sort(0, n-1, a);
    end=clock();
    double time= (double)(end-start)*1000/CLOCKS_PER_SEC;
    printf("Sorted array is: \n");
    for(int i=0; i<n; i++){
        printf("%d\t",a[i] );
    }
    printf("\nTime taken in ms : %f", time);
}
```

Output:

```
Enter the number of elements: 5

Enter element 1: 18

Enter element 2: 3

Enter element 3: 48

Enter element 4: 22

Enter element 5: 37
Sorted array is:
3       18      22      37      48
Time taken in ms : 0.000000
Process returned 0 (0x0)   execution time : 10.753 s
Press any key to continue.
```

Graph:



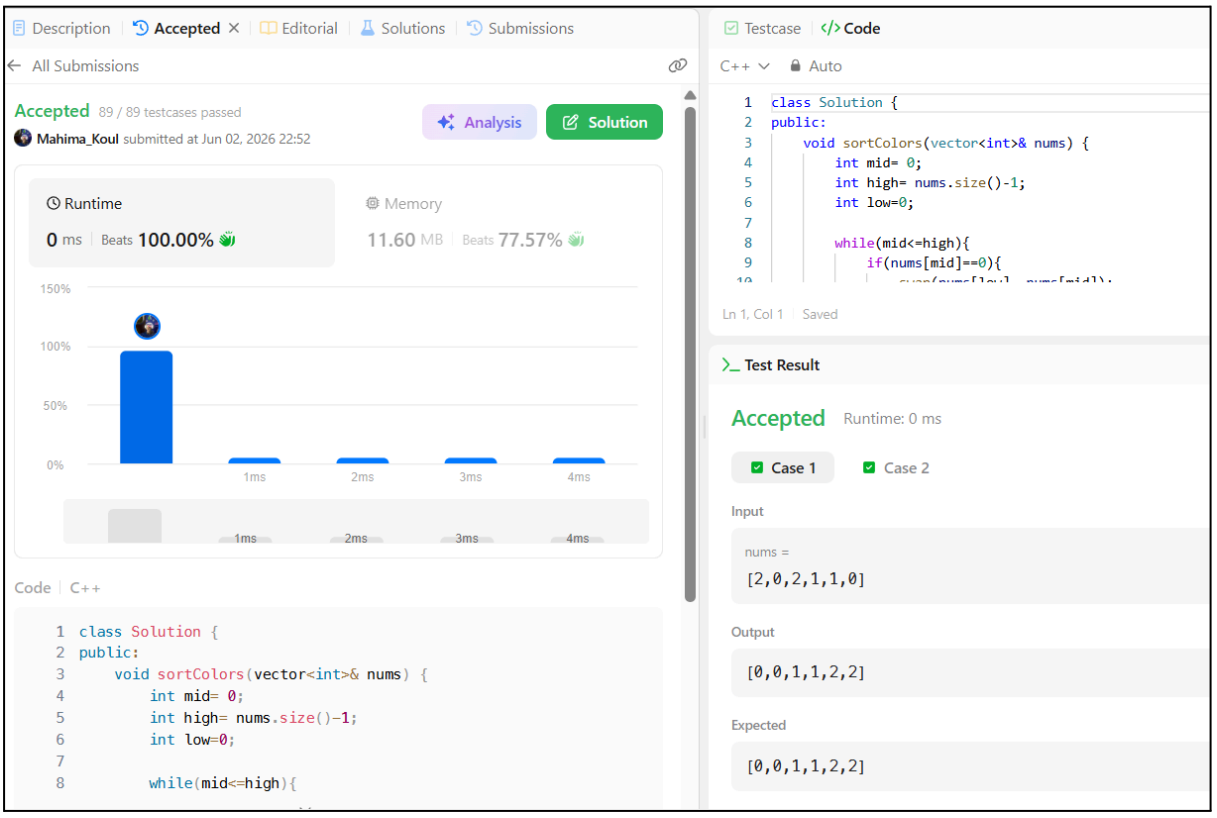
b) Leetcode Program related to sorting

Q: Leetcode 75:Given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.We will use the integers 0, 1, and 2 to represent the colors red, white, and blue, respectively. You must solve this problem without using the library's sort function.

Code:

```
void sortColors(int* nums, int numsSize) {
    int low = 0, mid = 0, high = numsSize - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {
            int temp = nums[low];
            nums[low] = nums[mid];
            nums[mid] = temp;
            low++;
            mid++;
        }
        else if (nums[mid] == 1) {
            mid++;
        }
        else {
            int temp = nums[mid];
            nums[mid] = nums[high];
            nums[high] = temp;
            high--;
        }
    }
}
```

Output:


Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 89 / 89 testcases passed
Mahima_Koul submitted at Jun 02, 2026 22:52
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
11.60 MB | Beats 77.57%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Code | C++
class Solution {
public:
void sortColors(vector<int>& nums) {
int mid= 0;
int high= nums.size()-1;
int low=0;
while(mid<=high){
Testcase
Code
C++
Auto
Ln 1, Col 1
Saved
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[2,0,2,1,1,0]
Output
[0,0,1,1,2,2]
Expected
[0,0,1,1,2,2]

LAB PROGRAM 4:

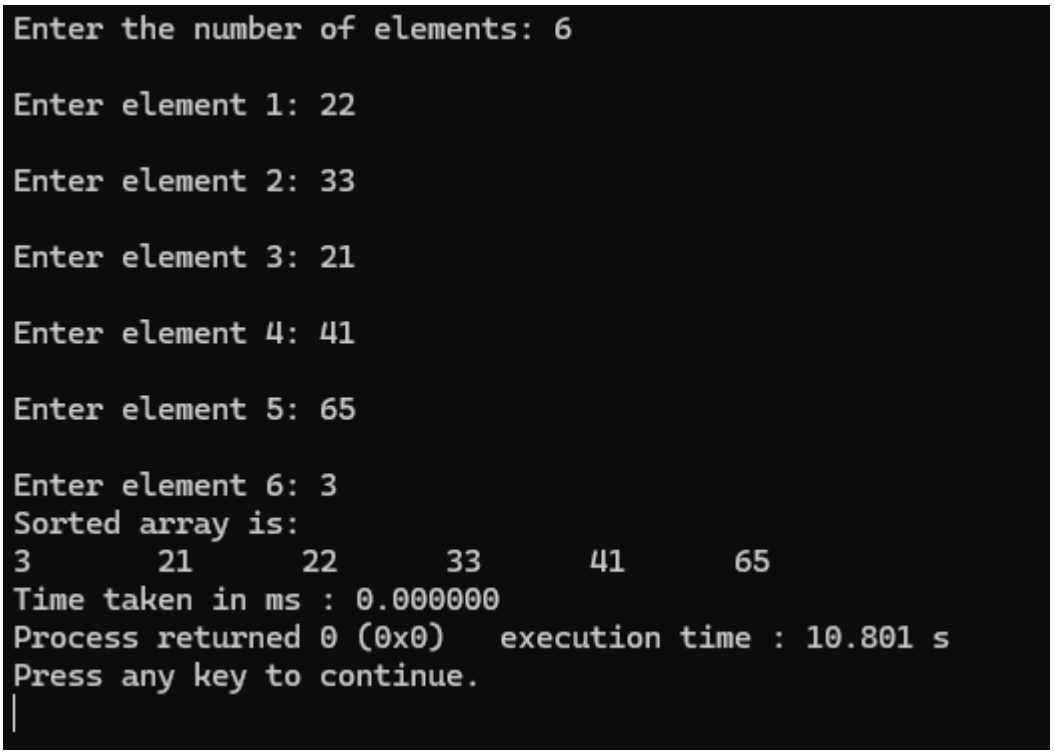
a) Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include<stdio.h>
#include<time.h>

void swap(int *a , int *b)
{
   int *temp= a;
   *a=*b;
   *b= *temp;

}

int partition(int low, int high, int a[])
{
   int i= low;
   int j= high+1;
   int pivot = a[low];
   while(i<j)
   {
      do
      {
         i=i+1;
      }while(pivot>=a[i]);
      do
      {
         j=j-1;
      }while(pivot < a[j]);

      if(i<j)
      {
         swap(&a[i], &a[j]);
      }
   }
   swap(&a[low], &a[j]);
   return j;
}

void quick_sort(int low, int high, int a[])
{
   if(low< high)
   {
      int j= partition(low, high, a);
      quick_sort(low, j-1, a);
      quick_sort(j+1, high, a);
   }
}

int main()
{
```

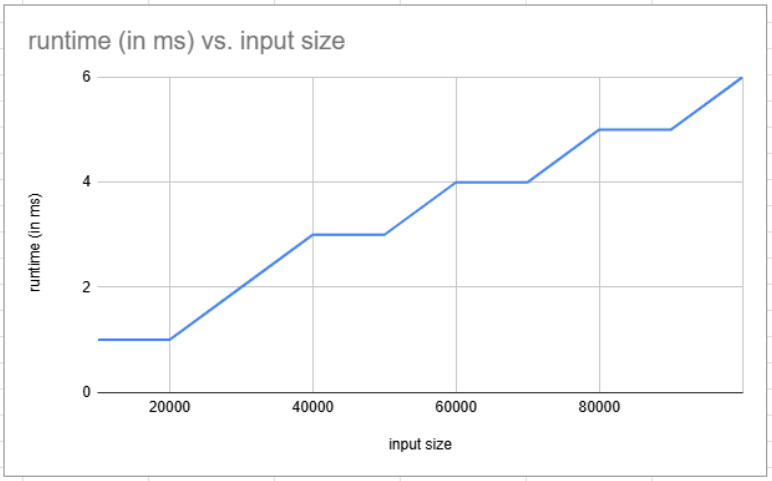
```
    int a[100000], n, i;
    clock_t start, end;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    for(int i=0; i<n; i++){
        printf("\nEnter element %d: ", i+1);
        scanf("%d", &a[i]);
        //a[i]= rand()%10000;
    }
    start=clock();
    quick_sort(0, n-1, a);
    end=clock();
    double time= (double)(end-start)*1000/CLOCKS_PER_SEC;
    printf("Sorted array is: \n");
    for(int i=0; i<n; i++){
        printf("%d\t",a[i] );
    }
    printf("\nTime taken in ms : %f", time);
}
```

Output:



Graph:

input size	runtime (in ms)
10000	1
20000	1
30000	2
40000	3
50000	3
60000	4
70000	4
80000	5
90000	5
100000	6



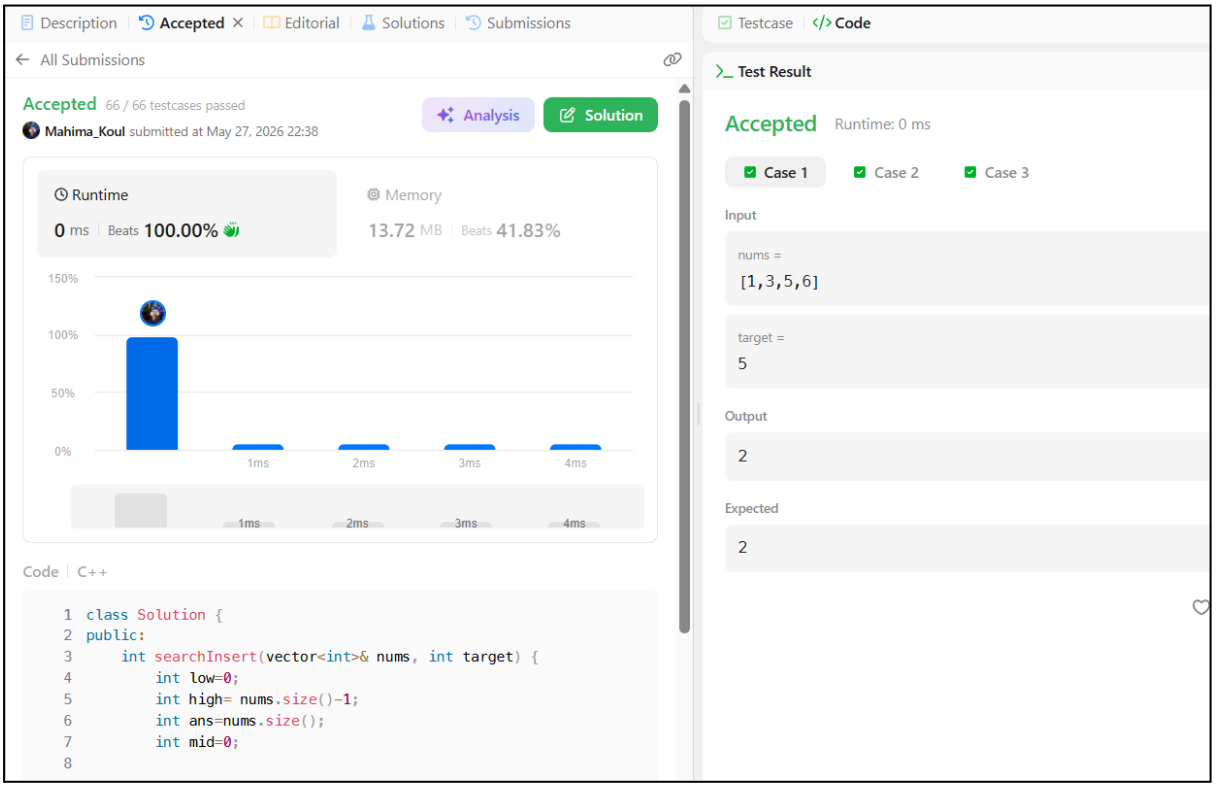
b) LeetCode Program related to sorting.

Q: LeetCode 35: Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order. You must write an algorithm with O(log n) runtime complexity.

Code:

```
int searchInsert(int* nums, int numsSize, int target) {
    int left = 0;
    int right = numsSize - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target)
            return mid;
        else if (nums[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return left;
}
```

Output:


Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 66 / 66 testcases passed
Mahima_Koul submitted at May 27, 2026 22:38
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
13.72 MB | Beats 41.83%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Code | C++
1 class Solution {
2 public:
3 int searchInsert(vector<int>& nums, int target) {
4 int low=0;
5 int high= nums.size()-1;
6 int ans=nums.size();
7 int mid=0;
8
Testcase
Code
Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,3,5,6]
target =
5
Output
2
Expected
2

LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

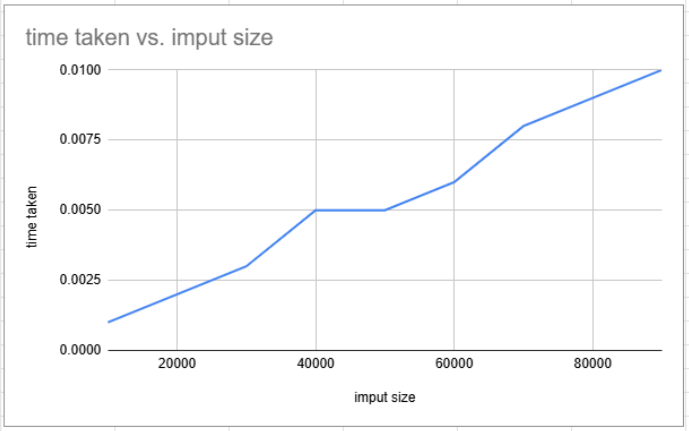
```
    int arr[n];

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    clock_t start, end;

    start = clock();
    heapSort(arr, n);
    end = clock();

    printf("Sorted array:\n");
    printArray(arr, n);

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time taken: %f seconds\n", time_taken);

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements:
36
78
64
99
25
Sorted array:
25 36 64 78 99
Time taken: 0.000000 seconds

Process returned 0 (0x0)   execution time : 11.541 s
Press any key to continue.
```

Graph:

input size	time taken
10000	0.001
20000	0.002
30000	0.003
40000	0.005
50000	0.005
60000	0.006
70000	0.008
80000	0.009
90000	0.01



LAB PROGRAM 6:

a) Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>

int max(int a, int b) {
   return (a > b) ? a : b;
}

int main() {
   int n, W;

   printf("Enter number of items: ");
   scanf("%d", &n);

   int val[n], wt[n];

   printf("Enter values of items:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &val[i]);

   printf("Enter weights of items:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &wt[i]);

   printf("Enter knapsack capacity: ");
   scanf("%d", &W);

   int dp[n + 1][W + 1];

   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= W; w++) {
         if (i == 0 || w == 0)
            dp[i][w] = 0;
         else if (wt[i - 1] <= w)
            dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                           dp[i - 1][w]);
         else
            dp[i][w] = dp[i - 1][w];
      }
   }

   printf("Maximum value = %d\n", dp[n][W]);

   int selected[n];
   int count = 0;
   int w = W;

   for (int i = n; i > 0 && w > 0; i--) {
      if (dp[i][w] != dp[i - 1][w]) {
         selected[count++] = i;
         w -= wt[i - 1];
```

```
        }
    }

    printf("Selected items:\n");

    for (int i = count - 1; i >= 0; i--) {
        int idx = selected[i] - 1;
        printf("Item %d (Value = %d, Weight = %d)\n",
               selected[i], val[idx], wt[idx]);
    }

    return 0;
}
```

Output:

```
Enter number of items: 4
Enter values of items:
8 6 16 11
Enter weights of items:
2 1 3 2
Enter knapsack capacity: 5
Maximum value = 27
Selected items:
Item 3 (Value = 16, Weight = 3)
Item 4 (Value = 11, Weight = 2)

Process returned 0 (0x0)   execution time : 13.114 s
Press any key to continue.
```

b) LeetCode Program related to Knapsack problem or Dynamic Programming.

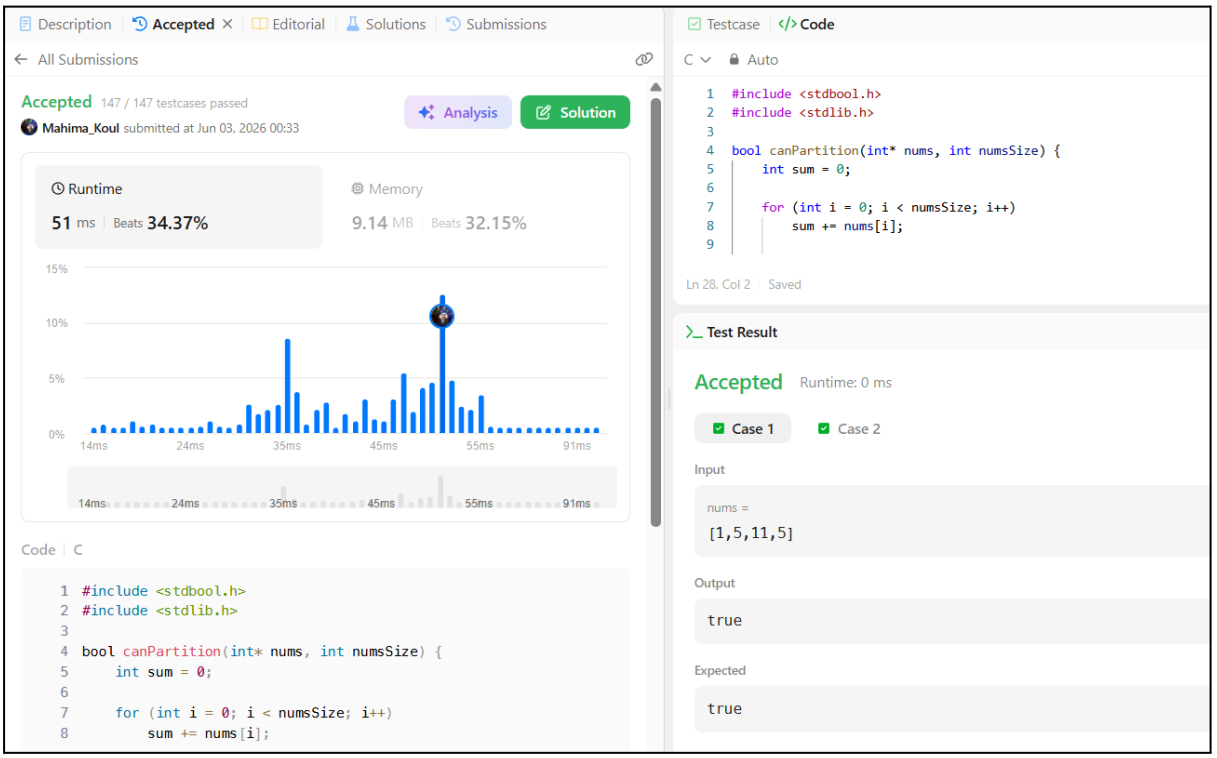
Q: LeetCode 416: Partition Equal Subset Sum - Given an integer array nums, return true if you can partition the array into two subsets such that the sum of the elements in both subsets is equal, or false otherwise.

Code:

```
#include <stdbool.h>
#include <stdlib.h>

bool canPartition(int* nums, int numsSize) {
    int sum = 0;

    for (int i = 0; i < numsSize; i++)
        sum += nums[i];

    if (sum % 2 != 0)
        return false;
```

```
    int target = sum / 2;

    bool* dp = (bool*)calloc(target + 1, sizeof(bool));
    dp[0] = true;

    for (int i = 0; i < numsSize; i++) {
        for (int j = target; j >= nums[i]; j--) {
            dp[j] = dp[j] || dp[j - nums[i]];
        }
    }

    bool result = dp[target];
    free(dp);

    return result;
}
```

Output:



LAB PROGRAM 7:

a) Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include <stdio.h>

#define INF 99999

void floydWarshall(int V, int dist[V][V]) {
   for (int k = 0; k < V; k++) {
      for (int i = 0; i < V; i++) {
         for (int j = 0; j < V; j++) {
            if (dist[i][k] != INF &&
               dist[k][j] != INF &&
               dist[i][k] + dist[k][j] < dist[i][j]) {

               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }
}

void printMatrix(int V, int dist[V][V]) {
   printf("\nShortest Distance between each pair:\n");

   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         if (dist[i][j] == INF)
            printf("INF ");
         else
            printf("%3d ", dist[i][j]);
      }
      printf("\n");
   }
}

int main() {
   int V;

   printf("Enter number of vertices: ");
   scanf("%d", &V);

   int dist[V][V];

   printf("Enter the adjacency matrix:\n");
   printf("(Use %d for no direct edge)\n", INF);

   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         scanf("%d", &dist[i][j]);
      }
   }
```

```
    floydWarshall(V, dist);

    printMatrix(V, dist);

    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter the adjacency matrix:
(Use 99999 for no direct edge)
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

Shortest Distance between each pair:
  0   5   8   9
INF   0   3   4
INF INF   0   1
INF INF INF   0

Process returned 0 (0x0)   execution time : 68.087 s
Press any key to continue.
```

b) LeetCode Program related to shortest distance calculation.

Q: LeetCode 743: Network Delay Time - You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = [ui, vi, wi], where ui is the source node, vi is the target node, and wi is the time it takes for a signal to travel from source to target. We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

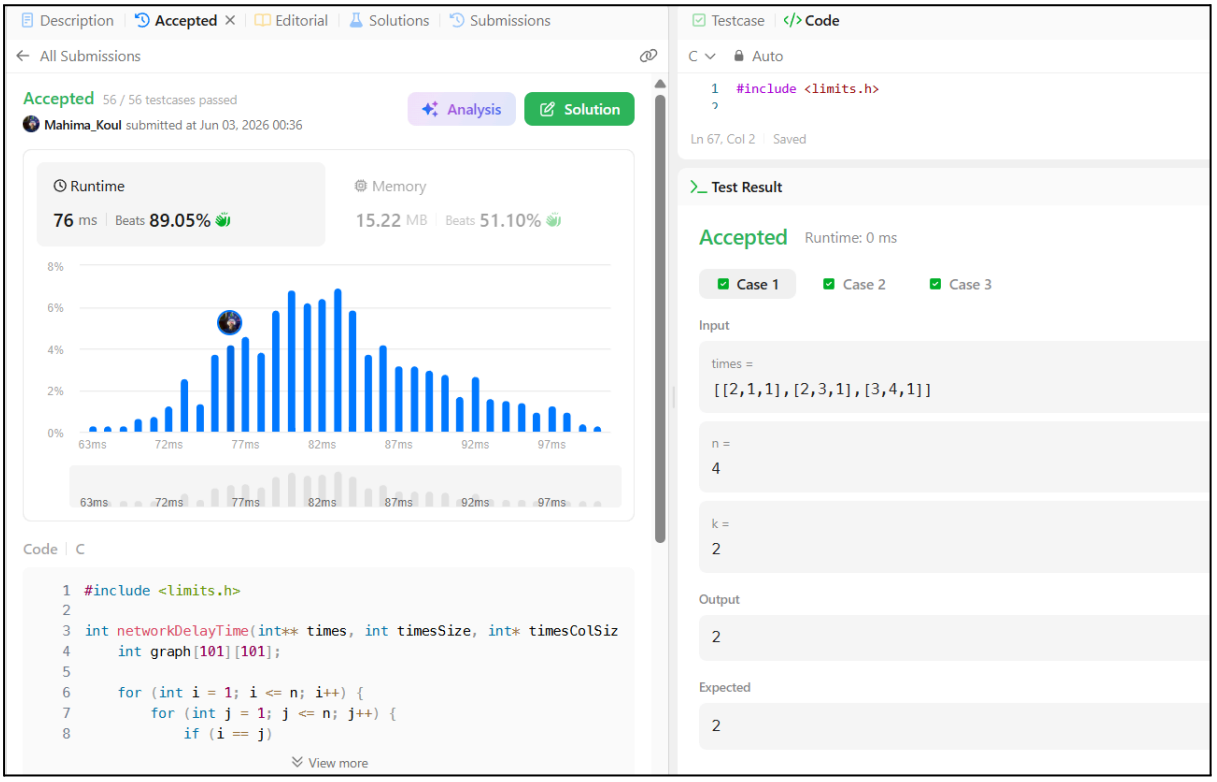
Code:

```
#include <limits.h>
int networkDelayTime(int** times, int timesSize, int* timesColSize, int n, int k) {
    int graph[101][101];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j)
                graph[i][j] = 0;
```

```
        else
            graph[i][j] = INT_MAX;
    }
}

for (int i = 0; i < timesSize; i++) {
    int u = times[i][0];
    int v = times[i][1];
    int w = times[i][2];
    graph[u][v] = w;
}

int dist[101];
int visited[101] = {0};

for (int i = 1; i <= n; i++)
    dist[i] = INT_MAX;

dist[k] = 0;

for (int count = 1; count <= n; count++) {
    int u = -1;
    int minDist = INT_MAX;

    for (int i = 1; i <= n; i++) {
        if (!visited[i] && dist[i] < minDist) {
            minDist = dist[i];
            u = i;
        }
    }

    if (u == -1)
        break;

    visited[u] = 1;

    for (int v = 1; v <= n; v++) {
        if (!visited[v] &&
            graph[u][v] != INT_MAX &&
            dist[u] != INT_MAX &&
            dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

int answer = 0;

for (int i = 1; i <= n; i++) {
    if (dist[i] == INT_MAX)
        return -1;

    if (dist[i] > answer)
        answer = dist[i];
}
```

```
    return answer;
}
```

Output:



LAB PROGRAM 8:

a)Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Code:

```
#include <stdio.h>
#define MAX 100
#define INF 999999

int main() {
   int n, cost[MAX][MAX];
   int visited[MAX] = {0};
   int minCost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix:\n");

   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0 && i != j) {
            cost[i][j] = INF;
         }
      }
   }

   visited[0] = 1; // Start from vertex 0

   int edges = 0;

   printf("\nEdges in Minimum Spanning Tree:\n");

   while (edges < n - 1) {
      int min = INF;
      int u = -1, v = -1;

      for (int i = 0; i < n; i++) {
         if (visited[i]) {
            for (int j = 0; j < n; j++) {
               if (!visited[j] && cost[i][j] < min) {
                  min = cost[i][j];
                  u = i;
                  v = j;
               }
            }
         }
      }

      if (u != -1 && v != -1) {
         printf("%d -> %d  Cost = %d\n", u, v, min);
         minCost += min;
```

```
            visited[v] = 1;
            edges++;
        }
    }
    printf("\nMinimum Cost = %d\n", minCost);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 10 15 30
10 0 40 20
15 40 0 50
30 20 50 0

Edges in Minimum Spanning Tree:
0 -> 1  Cost = 10
0 -> 2  Cost = 15
1 -> 3  Cost = 20

Minimum Cost = 45

Process returned 0 (0x0)   execution time : 3.783 s
Press any key to continue.
```

b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 30

typedef struct {
    int u, v, w;
} Edge;

Edge edge[MAX], temp;

int parent[MAX];

int find(int i) {
    while(parent[i])
        i = parent[i];
    return i;
}
```

```
int uni(int i, int j) {
   if(i != j) {
      parent[j] = i;
      return 1;
   }
   return 0;
}

int main() {
   int i, j, k, a, b, u, v, n, e;
   int mincost = 0;

   printf("Enter number of vertices and edges: ");
   scanf("%d%d", &n, &e);

   printf("Enter edges (u v w):\n");
   for(i = 0; i < e; i++) {
      scanf("%d%d%d", &edge[i].u, &edge[i].v, &edge[i].w);
   }

   // Sorting edges by weight
   for(i = 0; i < e - 1; i++) {
      for(j = 0; j < e - i - 1; j++) {
         if(edge[j].w > edge[j + 1].w) {
            temp = edge[j];
            edge[j] = edge[j + 1];
            edge[j + 1] = temp;
         }
      }
   }

   printf("\nEdges in MST:\n");

   for(i = 0; i < e; i++) {
      u = find(edge[i].u);
      v = find(edge[i].v);

      if(uni(u, v)) {
         printf("%d -- %d == %d\n",
               edge[i].u,
               edge[i].v,
               edge[i].w);

         mincost += edge[i].w;
      }
   }

   printf("\nMinimum cost = %d\n", mincost);

   return 0;
}
```

Output:

```
Enter number of vertices and edges: 4 5
Enter edges (u v w):
1 2 1
1 3 5
2 3 4
2 4 2
3 4 3

Edges in MST:
1 -- 2 == 1
2 -- 4 == 2
3 -- 4 == 3

Minimum cost = 6

Process returned 0 (0x0)   execution time : 33.021 s
Press any key to continue.
```

LAB PROGRAM 9:

a) Implement Fractional Knapsack using Greedy technique.

Code:

```
#include <stdio.h>
struct Item
{
   int value;
   int weight;
   float ratio;
};

void swap(struct Item *a, struct Item *b)
{
   struct Item temp= *a;
   *a=*b;
   *b= temp;
}

void sortItems(struct Item arr[], int n)
{
   for(int i=0; i<n-1; i++){
      for(int j=0; j<n-i-1; j++){
         if(arr[j].ratio < arr[j+1].ratio){
            swap(&arr[j], &arr[j+1]);
         }
      }
   }
}

float fractionalKnapsack(int capacity, struct Item arr[], int n)
{
   float totalValue= 0.0;
   for(int i=0; i<n; i++){
      if(capacity>= arr[i].weight){
         capacity -= arr[i].weight;
         totalValue += arr[i].value;
      }
      else{
         totalValue += arr[i].value* (float)capacity/arr[i].weight;
         break;
      }
   }
   return totalValue;
}

int main()
{
   int n, capacity;
   printf("Enter the number of items: ");
   scanf("%d", &n);

   struct Item arr[n];
```

```
    printf("\nEnter the value and weight of each item: ");
    for(int i=0; i<n; i++){
        scanf("%d%d", &arr[i].value, &arr[i].weight);
        arr[i].ratio= (float)arr[i].value/arr[i].weight;
    }
    printf("\nEnter capacity of knapsack :");
    scanf("%d", &capacity);

    sortItems(arr, n);

    float maxValue= fractionalKnapsack(capacity, arr , n);
    printf("\nThe maximum value of knapsack is: %.2f\n", maxValue);
    return 0;
}
```

Output:

```
Enter the number of items: 3

Enter the value and weight of each item: 60 10
100 20
120 30

Enter capacity of knapsack :50

The maximum value of knapsack is: 240.00

Process returned 0 (0x0)   execution time : 22.366 s
Press any key to continue.
```

b)LeetCode Program related to Greedy Technique algorithms.

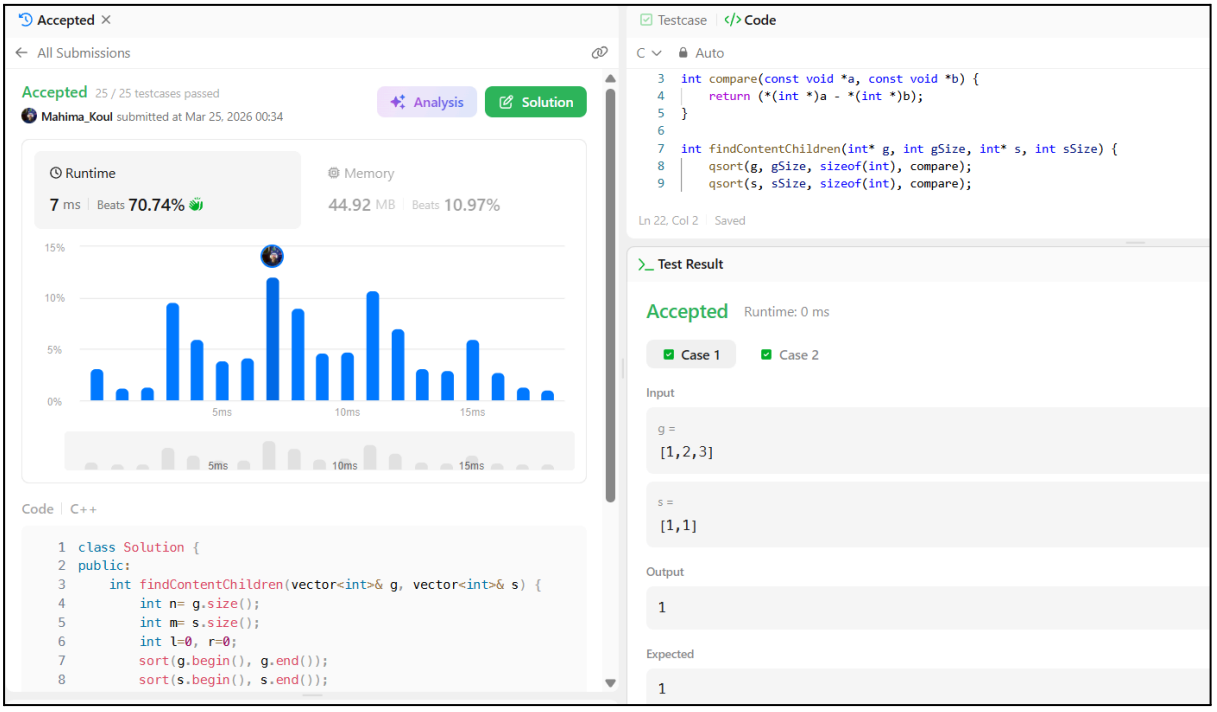
Q: LeetCode 455: Assign Cookies -Assume you are an awesome parent and want to give your children some cookies. Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with. Each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of content children and output the maximum number.

Code:

```
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}
```

```
int findContentChildren(int* g, int gSize, int* s, int sSize) {
    qsort(g, gSize, sizeof(int), compare);
    qsort(s, sSize, sizeof(int), compare);

    int child = 0;
    int cookie = 0;

    while (child < gSize && cookie < sSize) {
        if (s[cookie] >= g[child])
            child++;

        cookie++;
    }

    return child;
}
```

Output:



LAB PROGRAM 10:

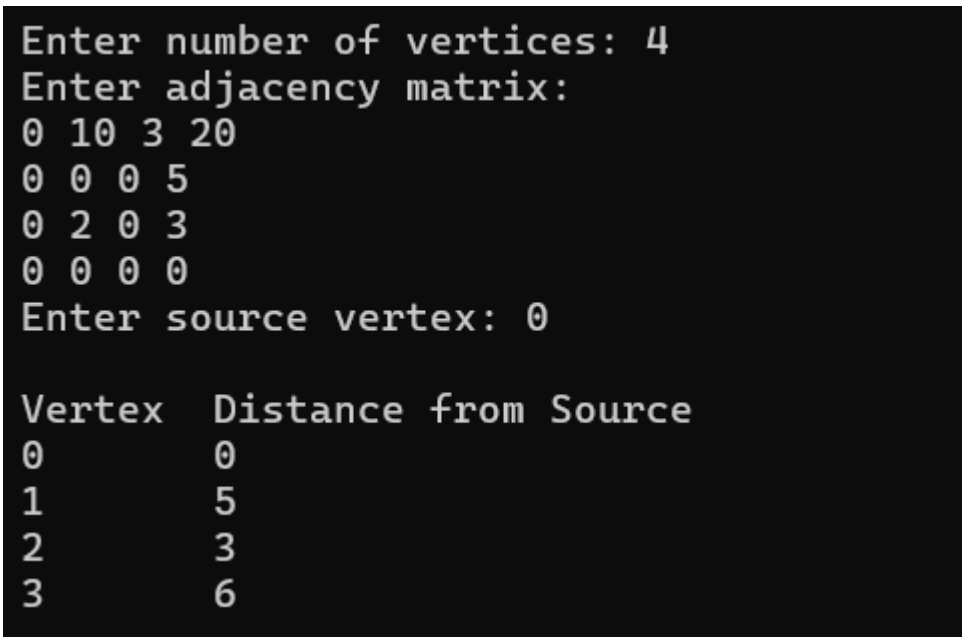
From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include <stdio.h>
#define MAX 10
#define INF 9999

int minDistance(int dist[], int visited[], int n)
{
   int min = INF, min_index = -1;
   for(int i = 0; i < n; i++)
   {
      if(visited[i] == 0 && dist[i] < min)
      {
         min = dist[i];
         min_index = i;
      }
   }
   return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int source)
{
   int dist[MAX];
   int visited[MAX];
   for(int i = 0; i < n; i++)
   {
      dist[i] = INF;
      visited[i] = 0;
   }

   dist[source] = 0;

   for(int count = 0; count < n - 1; count++)
   {
      int u = minDistance(dist, visited, n);
      visited[u] = 1;
      for(int v = 0; v < n; v++)
      {
         if(!visited[v] &&
            graph[u][v] &&
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v])
         {
            dist[v] = dist[u] + graph[u][v];
         }
      }
   }

   printf("\nVertex\tDistance from Source\n");
   for(int i = 0; i < n; i++)
```

```
    {
        printf("%d\t%d\n", i, dist[i]);
    }
}

int main()
{
    int n, source;
    int graph[MAX][MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);
    dijkstra(graph, n, source);
    return 0;
}
```

Output:



LAB PROGRAM 11:

Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int count = 0;
int isSafe(int **board, int row, int col, int n)
{
   int i, j;
   for (i = 0; i < row; i++)
   {
      if (board[i][col] == 1)
         return 0;
   }
   for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
   {
      if (board[i][j] == 1)
         return 0;
   }
   for (i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++)
   {
      if (board[i][j] == 1)
         return 0;
   }
   return 1;
}
void printSolution(int **board, int n)
{
   count++;
   printf("\nSolution %d:\n", count);
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         printf("%d ", board[i][j]);
      }
      printf("\n");
   }
}
void solveNQueens(int **board, int row, int n)
{
   if (row == n)
   {
      printSolution(board, n);
      return;
   }
   for (int col = 0; col < n; col++)
   {
      if (isSafe(board, row, col, n))
      {
         board[row][col] = 1;
         solveNQueens(board, row + 1, n);
```

```
            board[row][col] = 0;
        }
    }
}
int main()
{
    int n;
    printf("Enter board size (N): ");
    scanf("%d", &n);
    int **board = (int **)malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++)
    {
        board[i] = (int *)calloc(n, sizeof(int));
    }
    solveNQueens(board, 0, n);
    if (count == 0)
    {
        printf("\nNo solutions exist for N = %d\n", n);
    }
    else
    {
        printf("\nTotal Solutions = %d\n", count);
    }
    for (int i = 0; i < n; i++)
    {
        free(board[i]);
    }
    free(board);
    return 0;
}
```

Output:

```
Enter board size (N): 4

Solution 1:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Solution 2:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Total Solutions = 2

Process returned 0 (0x0)   execution time : 28.050 s
Press any key to continue.
```